Senior DevOps / SRE Interview Questions & Answers — Part 5

Advanced real-world DevOps/SRE interview preparation focused on Staff/Lead engineering expectations.

---

131. How do you troubleshoot high CPU throttling in Kubernetes?

Answer

CPU throttling usually happens when containers hit CPU limits.

I investigate:

```
kubectl top pod
kubectl describe pod
```

Then analyze:

- CPU requests vs limits
- burst traffic
- noisy neighbors
- application inefficiency
- sidecar overhead

I also check Prometheus metrics:

- container_cpu_cfs_throttled_seconds_total

Mitigation:

- tune requests/limits
- optimize application
- scale horizontally
- isolate workloads

Throttling often causes latency spikes before outages.

---

132. Explain Kubernetes ResourceQuota and LimitRange.

Answer

ResourceQuota

Limits total namespace resource consumption.

Example:

- max CPU
- max memory
- max pods

LimitRange

Defines default/min/max container limits.

Used to:

- prevent resource abuse
- improve cluster governance

These controls are essential in multi-tenant environments.

---

133. How do you debug slow DNS resolution in Kubernetes?

Answer

I validate:

- CoreDNS performance
- upstream resolvers
- network latency
- cache misses

Commands:

```
kubectl logs -n kube-system deployment/coredns
```

Metrics monitored:

- DNS request latency
- cache hit ratio
- NXDOMAIN spikes

Common causes:

- overloaded CoreDNS
- excessive queries
- external DNS slowness

---

134. Explain the difference between StatefulSet rolling update and Deployment rolling update.

Answer

Deployment updates pods in parallel based on strategy.

StatefulSet updates sequentially.

Reasons:

- preserve identity
- maintain quorum
- ensure ordered startup/shutdown

This is critical for databases and distributed systems.

---

135. How do you troubleshoot kubelet failures?

Answer

Checks:

```
systemctl status kubelet
journalctl -u kubelet
```

Common causes:

- disk full
- certificate expiration
- container runtime issues
- API server connectivity
- resource exhaustion

Kubelet instability can cause node-wide disruption.

---

136. Explain Kubernetes control plane components.

Answer

API Server

Central communication layer.

etcd

Cluster state store.

Scheduler

Assigns pods to nodes.

Controller Manager

Maintains desired state.

Cloud Controller Manager

Integrates cloud provider APIs.

Understanding control plane internals is essential for large-cluster troubleshooting.

---

137. How do you secure Terraform state files?

Answer

Terraform state often contains secrets.

My controls:

- encrypted backend
- S3 versioning
- IAM restrictions
- DynamoDB locking
- audit logging
- restricted CI access

State security is critical because compromise can expose infrastructure.

---

138. Explain Terraform modules and their benefits.

Answer

Modules improve:

- reusability
- consistency
- standardization
- governance

I design modules with:

- versioning
- input validation
- outputs
- environment abstraction

Good modules reduce operational drift.

---

139. How do you manage multi-environment Terraform deployments?

Answer

I isolate environments using:

- separate state files
- workspaces carefully
- environment-specific variables
- CI/CD pipelines

Critical production environments require stronger isolation.

---

140. Explain immutable tags vs latest tag.

Answer

I avoid latest tags in production.

Reasons:

- non-deterministic deployments
- rollback difficulty
- auditability issues

I prefer:

- semantic versioning
- image digests
- immutable artifacts

Reproducibility is critical for reliability.

---

141. How do you troubleshoot failing health checks in AWS Load Balancer?

Answer

I validate:

- target group health
- application response
- security groups
- ports
- readiness behavior

Common causes:

- wrong path
- timeout mismatch
- app startup delays
- dependency failures

Health checks must represent real application readiness.

---

142. Explain NAT Gateway bottlenecks and optimization.

Answer

NAT Gateways can become expensive and overloaded.

Common issues:

- excessive egress
- cross-AZ traffic
- large image pulls

Optimizations:

- VPC endpoints
- local caching
- ECR pull-through cache
- traffic localization

Cloud networking costs scale rapidly in large environments.

143. How do you debug IAM permission issues?

Answer

I validate:

- IAM policies
- trust relationships
- SCP restrictions
- role assumptions
- token expiration

Tools:

- CloudTrail
- IAM policy simulator

IAM debugging often requires tracing indirect denies.

---

144. Explain AWS security groups vs NACLs.

Answer

Security Groups

- stateful
- instance-level
- allow rules only

NACLs

- stateless
- subnet-level
- allow and deny rules

I primarily use security groups for workload security and NACLs for coarse network boundaries.

---

145. How do you secure container registries?

Answer

Controls:

- private registries
- RBAC
- image signing
- vulnerability scanning
- immutable tags
- audit logging

I also restrict public image usage.

Supply chain attacks increasingly target registries.

---

146. Explain sidecar injection failures.

Answer

Common causes:

- webhook failures
- namespace labels missing
- certificate problems
- API server latency

I validate:

- mutating webhook configs
- injector logs
- admission controller status

Service mesh onboarding often fails due to sidecar injection issues.

---

147. How do you troubleshoot Envoy sidecar latency?

Answer

I check:

- mTLS overhead
- retry storms
- circuit breaker config
- CPU saturation
- connection pools

Commands:

```
istioctl proxy-status
```

Sidecars improve observability/security but increase complexity and latency.

---

148. Explain API Gateway vs Load Balancer.

Answer

API Gateway

- authentication
- rate limiting
- API management
- request transformation

Load Balancer

- traffic distribution
- L4/L7 routing
- high availability

API Gateways add governance and control.

---

149. How do you debug Redis latency issues?

Answer

Checks:

- memory pressure
- slow commands
- eviction policy
- network latency
- replication lag

Commands:

```
redis-cli slowlog get
```

Redis problems often cascade into application latency.

---

150. Explain distributed tracing and why it matters.

Answer

Distributed tracing follows requests across services.

Benefits:

- latency analysis
- dependency mapping
- bottleneck detection
- retry visibility

Tools:

- Jaeger
- Tempo
- OpenTelemetry

Tracing is essential for debugging microservices.

---

151. How do you handle production schema migrations safely?

Answer

I use backward-compatible migrations.

Strategies:

- expand-contract pattern
- feature flags
- phased rollout
- migration testing

Database changes are among the highest-risk production operations.

---

152. Explain rolling restart vs rollout restart.

Answer

Rolling restart updates pods gradually without changing configuration.

Useful for:

- picking up secrets/configs
- refreshing workloads

Commands:

```
kubectl rollout restart deployment
```

---

153. How do you prevent cascading failures?

Answer

Strategies:

- circuit breakers
- retries with backoff

- bulkheads
- rate limiting
- queue buffering
- graceful degradation

The key principle:

Contain failures before they spread.

---

154. Explain graceful degradation.

Answer

Graceful degradation means partial functionality instead of total outage.

Examples:

- cached responses
- disabling recommendations
- read-only mode

This improves user experience during incidents.

---

155. How do you troubleshoot thread exhaustion?

Answer

Symptoms:

- request queue buildup
- timeouts
- latency spikes

I investigate:

- blocked threads
- slow downstreams
- deadlocks
- connection pools

Tools:

- thread dumps

- APM traces

Thread exhaustion often appears as infrastructure instability.

---

156. Explain blue-green database deployment challenges.

Answer

Main challenge:

Database state synchronization.

Risks:

- incompatible schemas
- replication lag
- partial writes

I use:

- backward-compatible migrations
- phased traffic shift
- rollback-safe schema design

Databases make blue-green deployments significantly harder.

---

157. How do you optimize Prometheus performance?

Answer

Optimizations:

- reduce cardinality
- tune retention
- shard workloads
- recording rules
- remote write
- efficient scrape intervals

Prometheus scaling becomes challenging in very large clusters.

---

158. Explain SLI, SLO, and SLA.

Answer

SLI

Actual measured metric.

SLO

Target reliability objective.

SLA

Business/legal commitment.

SLOs should drive operational decisions.

---

159. How do you design self-healing systems?

Answer

Self-healing requires:

- health checks
- autoscaling
- automated rollback
- failover automation
- reconciliation loops

But automation must be carefully controlled to avoid amplifying failures.

---

160. What defines operational excellence in DevOps/SRE?

Answer

Operational excellence means:

- fast recovery
- reliable deployments
- low toil
- strong observability
- clear ownership
- automation maturity
- resilience culture

The goal is not avoiding failures entirely.

The goal is recovering quickly, learning continuously, and preventing repeat incidents.